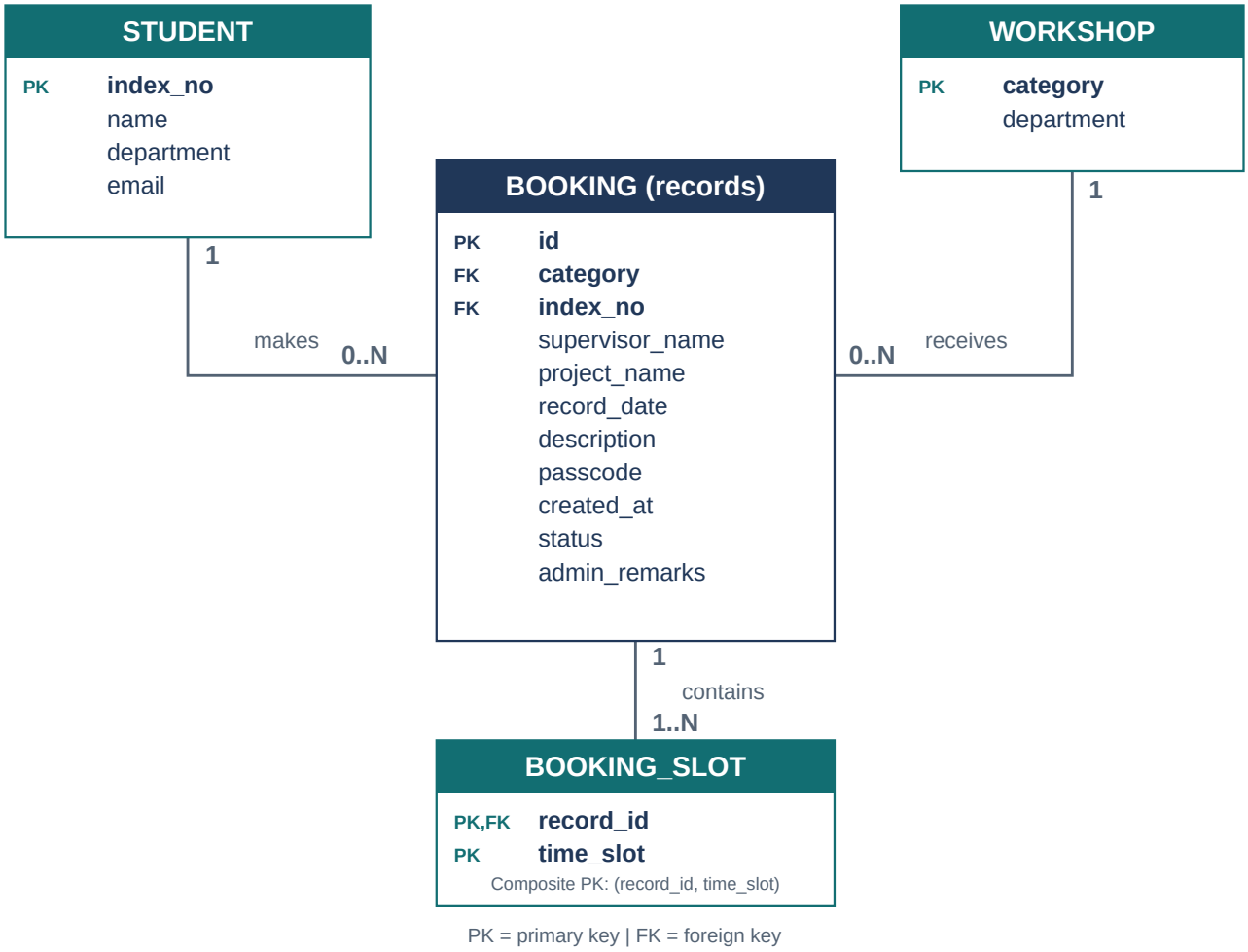
SYNCLAB - ER Diagram

Entities, attributes, relationships and cardinalities of the implemented SQLite database.



Relationships and cardinalities

Relationship	Cardinality	Meaning
STUDENT makes BOOKING	1 : 0..N	A student can have zero or many bookings. Each booking belongs to exactly one student.
WORKSHOP receives BOOKING	1 : 0..N	A workshop can have zero or many bookings. Each booking uses exactly one workshop.
BOOKING contains BOOKING_SLOT	1 : 1..N	A booking has one or more selected slots in the application. Each slot membership belongs to exactly one booking.

Entity mapping: STUDENT = students; WORKSHOP = workshops; BOOKING = records; BOOKING_SLOT = booking_slots.

Minimum slot count: The application requires at least one slot when saving a booking. The SQL foreign key alone allows a booking to have zero slot rows. The diagram shows the application rule.

Relational Schema

Exact table and attribute names used in the application. PK = primary key; FK = foreign key.

workshops (WORKSHOP)

Attribute	SQLite type	Key	Nullable
category	TEXT	PK	No
department	TEXT	-	No

students (STUDENT)

Attribute	SQLite type	Key	Nullable
index_no	TEXT	PK	No
name	TEXT	-	No
department	TEXT	-	No
email	TEXT	-	No

records (BOOKING)

Attribute	SQLite type	Key	Nullable
id	INTEGER	PK	No
category	TEXT	FK	No
index_no	TEXT	FK	No
supervisor_name	TEXT	-	No
project_name	TEXT	-	No
record_date	TEXT	-	No
description	TEXT	-	Yes
passcode	TEXT	-	No
created_at	TEXT	-	No
status	TEXT	-	No
admin_remarks	TEXT	-	Yes

booking_slots (BOOKING_SLOT)

Attribute	SQLite type	Key	Nullable
record_id	INTEGER	PK (part 1), FK	No
time_slot	TEXT	PK (part 2)	No

Foreign key references

records.category -> workshops.category
records.index_no -> students.index_no
booking_slots.record_id -> records.id **(ON DELETE CASCADE)**

Key rules: booking_slots has one composite primary key: **(record_id, time_slot)**. records.id is an auto-incrementing integer. passcode is not declared UNIQUE in the database.

Other constraints: status defaults to Pending and accepts Pending, Approve or Reject. time_slot must contain non-whitespace text and cannot contain a comma. record_date stores an ISO date (YYYY-MM-DD) as TEXT.

Source: database/databasemanager.cpp, schemaStatements() and validateRecord(); sql/create_tables.sql. Student and workshop departments are separate attributes.

Normalization and SQL

Short note on normalization decisions

The database is designed up to Fourth Normal Form (4NF) under the stated booking dependencies. Repeating time-slot groups were removed by storing each selected slot as a separate row in booking_slots, giving atomic values for 1NF. All non-key attributes depend on the complete primary key, satisfying 2NF. Student details and workshop departments are stored in separate tables, removing transitive dependencies for 3NF. Every nontrivial functional dependency has a candidate-key determinant, satisfying BCNF. Booking facts and slot memberships are separated, and no independent multivalued collections are combined in one table, satisfying 4NF under these dependencies. The composite key (record_id, time_slot) prevents duplicate slots within one booking.

Functional dependencies: category determines workshop department; index_no determines student name, department and email; id determines the single-valued booking attributes.

CREATE TABLE statements

File: sql/create_tables.sql. Execute first in a new, empty SQLite database.

```
-- SYNCLAB: four-table 4NF schema for a NEW demonstration database.
-- Run this file first, then sample_data_small.sql. The app migrates saved data.
PRAGMA foreign_keys = ON;
BEGIN IMMEDIATE;

CREATE TABLE workshops (
    category TEXT NOT NULL PRIMARY KEY,
    department TEXT NOT NULL
);

CREATE TABLE students (
    index_no TEXT NOT NULL PRIMARY KEY,
    name TEXT NOT NULL,
    department TEXT NOT NULL,
    email TEXT NOT NULL
);

CREATE TABLE records (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    category TEXT NOT NULL REFERENCES workshops(category),
    index_no TEXT NOT NULL REFERENCES students(index_no),
    supervisor_name TEXT NOT NULL,
    project_name TEXT NOT NULL,
    record_date TEXT NOT NULL,
    description TEXT,
    passcode TEXT NOT NULL,
    created_at TEXT NOT NULL,
    status TEXT NOT NULL DEFAULT 'Pending'
        CHECK (status IN ('Pending', 'Approve', 'Reject')),
    admin_remarks TEXT
);

CREATE TABLE booking_slots (
    record_id INTEGER NOT NULL REFERENCES records(id) ON DELETE CASCADE,
    time_slot TEXT NOT NULL
        CHECK (length(trim(time_slot)) > 0 AND instr(time_slot, ',') = 0),
    PRIMARY KEY (record_id, time_slot)
);

CREATE INDEX records_category_date ON records(category, record_date);
PRAGMA user_version = 4;
COMMIT;
```

Sample Data INSERT Script

File: sql/sample_data_small.sql. Execute after create_tables.sql. All student, supervisor and project details are fictional.

```
-- Fictional sample data. Run once after create_tables.sql in an empty demo DB.
-- Student DEMO-001 has two bookings; booking 1 has two separate slot rows.
PRAGMA foreign_keys = ON;
BEGIN IMMEDIATE;

INSERT INTO workshops (category, department) VALUES
('Computer', 'Department of Nano Science Technology'),
('Manufacturing', 'Department of Mechanical and Manufacturing Technology');

INSERT INTO students (index_no, name, department, email) VALUES
('DEMO-001', 'Demo Student One', 'Department of Construction Technology',
 'student.one@example.org'),
('DEMO-002', 'Demo Student Two', 'Department of Nano Science Technology',
 'student.two@example.org');

INSERT INTO records (
    id, category, index_no, supervisor_name, project_name, record_date,
    description, passcode, created_at, status, admin_remarks
) VALUES
(1, 'Manufacturing', 'DEMO-001', 'Demo Supervisor One', 'Component Fabrication',
 '2030-01-12', 'Manufacturing practical with two time slots.', '2001',
 '2030-01-05T10:00:00', 'Approve', 'Approved for the practical.'),
(2, 'Computer', 'DEMO-002', 'Demo Supervisor Two', 'Programming Practical',
 '2030-01-12', 'Programming laboratory session.', '2002',
 '2030-01-05T10:15:00', 'Pending', NULL),
(3, 'Computer', 'DEMO-001', 'Demo Supervisor One', 'Data Analysis',
 '2030-01-13', 'Second booking by the same student.', '2003',
 '2030-01-05T10:30:00', 'Reject', 'Reschedule this session.');

INSERT INTO booking_slots (record_id, time_slot) VALUES
(1, '8.30 am - 10.30 am'),
(1, '10.30 am - 12.30 pm'),
(2, '1.30 pm - 3.30 pm'),
(3, '8.30 am - 10.30 am');

COMMIT;
```

What the sample demonstrates

There are **two workshops, two students, three bookings and four slot rows**. DEMO-001 has two bookings but only one student profile. Booking 1 has two separate slot rows, demonstrating removal of the repeating group. The sample uses Computer and Manufacturing from the current workshop catalog.

Execution order: create_tables.sql, then sample_data_small.sql. The schema is the same as the application database. These scripts are intended for a new demonstration database; the application handles migration of saved data.

Validation: Both complete scripts execute successfully in SQLite. Foreign keys, composite-key uniqueness, status and slot checks, and cascading slot deletion were verified.